

from Beginner to Master

# DevOps를 위한 **Docker** 가상화

## Section 8. Logging and Monitoring

```
book:
  def setData(self, title, price, author):
    self.title = title
    self.price = price
    self.author = author

var fs = require('fs');
fs.readFile('./ONE.txt', '* 1 *',
  function (err, data) {
    console.log(data); // 3
  });
ApplicationDelegate > {
```

```
public static void main(String[] args)

<servlet>
  <servlet-name>BoardController</servlet-name>
  <servlet-class>com.joneconsulting.controller.BoardCont
  <init-param>
    <param-name>user_name</param-name>
    <param-value>henneth Lee</param-value>
  </init-param>
</servlet>
```

Winterface

Dowon Lee

수강생 19K 강의평점 4.8(1K)



멘토링 ON

멘토링 설정

- 홈
- 강의
- 로드맵
- 수강평
- 게시글
- 블로그

수정하기

강의 전체 6



[개정판] 웹 애플리케이션 개발을 위한 IntelliJ IDEA 설정

▶ 학습중

★ 0강 / 25강 (0%)

818명



멀티OS 사용을 위한 가상화 환경 구축 가이드 (Docker + Kubernetes)

▶ 학습중

★ 0강 / 16강 (0%)

924명



Jenkins를 이용한 CI/CD Pipeline 구축

▶ 학습중

★ 81강 / 83강 (98%)

독점 2709명

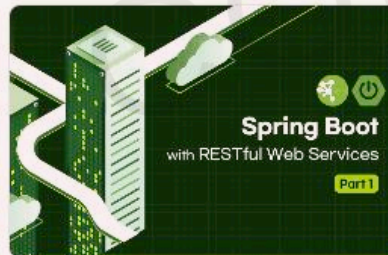


Spring Cloud로 개발하는 마이크로서비스 애플리케이션(MSA)

▶ 학습중

★ 156강 / 156강 (100%)

독점 5531명



Spring Boot를 이용한 RESTful Web Services 개발

▶ 학습중

★ 3강 / 48강 (6%)

독점 3862명



[구버전] 웹 애플리케이션 개발을 위한 IntelliJ IDEA 설정 (2020 ver.)

▶ 학습중

★ 14강 / 14강 (100%)

4813명

이 되었던 때가 있었습니  
"IT 엔지니어"라고 적고

을 가지고 있습니다. 누구  
사람입니다. 개발자로서, 강  
지만, 그래도, 남들보다 조

ecture, Blockchain,

- Section 0: Introduction to Cloud Native Technologies
- Section 1: Docker Essentials - Container
- Section 2: Docker Essentials - Image
- Section 3: Docker Network and Storage
- Section 4: Building and Managing Containerized Application
- Section 5: Container Orchestration
- Section 6: Continuous Integration and Continuous Deployment
- Section 7: Docker Security
- **Section 8: Logging and Monitoring**
- Section 9: Advanced Docker Usage
- Section 10: Capstone Project

Section 8.

# Logging and Monitoring

- Docker Container Logging
- Docker Container Monitoring



# Docker Container Logging

## Logging

- 도커 내부에서 발생하는 로그 메시지 확인

| 도커 실행 시 또는 실행 이후에 발생 되는 로그 메시지 출력

```
$ docker logs <Container ID or Name>
```

| mariadb 실행에 대한 로그 확인

```
$ docker run -d -p 13306:3306 \  
    -e MARIADB_ALLOW_EMPTY_ROOT_PASSWORD=true \  
    --name mariadb1 \  
    mariadb:latest
```

```
$ docker logs mariadb1
```

# Docker Container Logging

## Logging (Linux)

- 컨테이너 로그 파일 확인

- | docker inspect → "LogPath": "/var/lib/docker/containers/~
  - | /var/lib/docker/containers/\${CONTAINER\_ID}/\${CONTAINER\_ID}-json.log

```
[ec2-user@ip-172-31-5-111 ~]$ docker ps -a
```

| CONTAINER ID | IMAGE          | COMMAND                  | CREATED       | STATUS       | PORTS               |
|--------------|----------------|--------------------------|---------------|--------------|---------------------|
| 7aa484a25939 | mariadb:latest | "docker-entrypoint.s..." | 7 minutes ago | Up 7 minutes | 0.0.0.0:13306->3306 |

```
[ec2-user@ip-172-31-5-111 ~]$ sudo ls -al /var/lib/docker/containers/7aa484a2593964038ce8c5e7bb86494bf23f4040293e3ab9ae17de6680c98bab-
total 40
drwx--x--- 4 root root  237 Apr 21 14:29 .
drwx--x--- 3 root root   78 Apr 21 14:29 ..
-rw-r----- 1 root root 14313 Apr 21 14:29 7aa484a2593964038ce8c5e7bb86494bf23f4040293e3ab9ae17de6680c98bab-json.log
```

```
$ sudo cat /var/lib/docker/containers/7aa484a~/7aa484a~-json.log
```

# Docker Container Logging

## Logging (Linux)

- 컨테이너 로그 파일 → syslog

| syslog → 유닉스 계열 운영체제에서 로그 수집하는 도구 (커널, 보안, 애플리케이션 로그 등)

```
$ docker run -d -p 13306:3306 --log-driver syslog  
-e MARIADB_ALLOW_EMPTY_ROOT_PASSWORD=true \  
--name mariadb1 \  
mariadb:latest
```

```
[ec2-user@ip-172-31-5-111 ~]$ docker run -d --rm --log-driver=syslog -p 13306:3306 -e MARIADB_ALLOW_EMPTY_ROOT_PASSWORD=true --name mariadb1 mariadb:latest  
c68c243acbb35b9c853e4947b556247857469f36a56c0cd02d8a27bb5366e79c  
[ec2-user@ip-172-31-5-111 ~]$ journalctl -u docker.service | grep c68c  
Apr 21 14:48:09 ip-172-31-5-111.ap-northeast-2.compute.internal dockerd[3101]: time="2024-04-21T14:48:09.291290636Z" level=info msg="Configured log driver does not support reads, enabling local file cache for container logs" container=c68c243acbb35b9c853e4947b556247857469f36a56c0cd02d8a27bb5366e79c driver=syslog  
Apr 21 14:48:09 ip-172-31-5-111.ap-northeast-2.compute.internal c68c243acbb3[3101]: 2024-04-21 14:48:09+00:00 [Note] [Entrypoint]: Entrypoint script for MariaDB Server 1:11.3.2+maria~ubu2204 started.  
Apr 21 14:48:09 ip-172-31-5-111.ap-northeast-2.compute.internal c68c243acbb3[3101]: 2024-04-21 14:48:09+00:00 [Warn] [Entrypoint]: /sys/fs/cgroup/freezer:/docker/c68c243acbb35b9c853e4947b556247857469f36a56c0cd02d8a27bb5366e79c  
Apr 21 14:48:09 ip-172-31-5-111.ap-northeast-2.compute.internal c68c243acbb3[3101]: 10:devices:/docker/c68c243acbb35b9c853e4947b556247857469f36a56c0cd02d8a27bb5366e79c
```

# Docker Container Logging

## Logging (Windows)

- 컨테이너 로그 파일 확인

- | docker inspect → "LogPath": "/var/lib/docker/containers/~  
| /var/lib/docker/containers/\${CONTAINER\_ID}/\${CONTAINER\_ID}-json.log

- Windows에서는 /var/lib/docker/containers/~ 의 내용 확인 어려움

- | /var/lib/docker/container를 Volume mount로 연결한 Linux Container에서 내용 확인 가능

```
C:\> docker run -v /:/data -it ubuntu /bin/bash
```

```
# chroot /data
```

```
# cd /var/lib/docker/containers
```

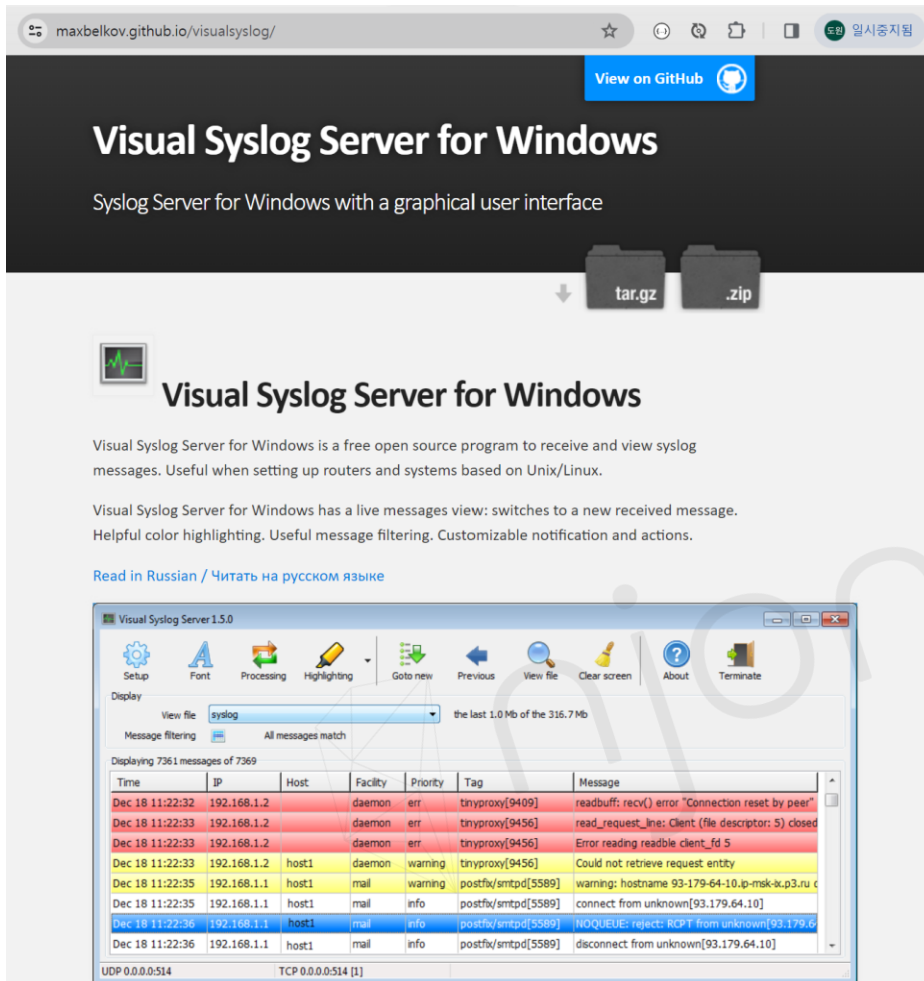
```
# ls -al
```

```
drwx--x---  4 root root  4096 Apr 22 14:13 9d19a5067fcbdb80d634b8a5e343d150ec9a7b09aa15d4da9fa291c7f40a769e
drwx--x---  4 root root  4096 Apr 22 14:13 b4526e85340fb2d2fad1cda441b7f332baa5e5bb0a3b1fb13b4ebb0b746ee3ac
drwx--x---  4 root root  4096 Apr 22 14:13 b89558bb32a825b1b317f28b93e51f5c504a3a96edc2d7a2b0202c3c49635826
drwx--x---  4 root root  4096 Apr 22 14:13 b99e9ba87df48130e64d5be668f3747c275534d867fbd61be6c226aa9a855316
drwx--x---  4 root root  4096 Apr 22 14:13 bfef3c20b11c50c4ac418e96c3bf89f0519c2041c50953e8e00172f5f723e218
```

# Syslog Server for Windows

## Visual Syslog

- <https://maxbelkov.github.io/visualsyslog/>



The screenshot shows the GitHub repository page for Visual Syslog Server for Windows. The page title is "Visual Syslog Server for Windows" and the subtitle is "Syslog Server for Windows with a graphical user interface". There are download buttons for "tar.gz" and ".zip". Below the download buttons, there is a section titled "Visual Syslog Server for Windows" with a description: "Visual Syslog Server for Windows is a free open source program to receive and view syslog messages. Useful when setting up routers and systems based on Unix/Linux." and "Visual Syslog Server for Windows has a live messages view: switches to a new received message. Helpful color highlighting. Useful message filtering. Customizable notification and actions." There is also a link to "Read in Russian / Читать на русском языке".

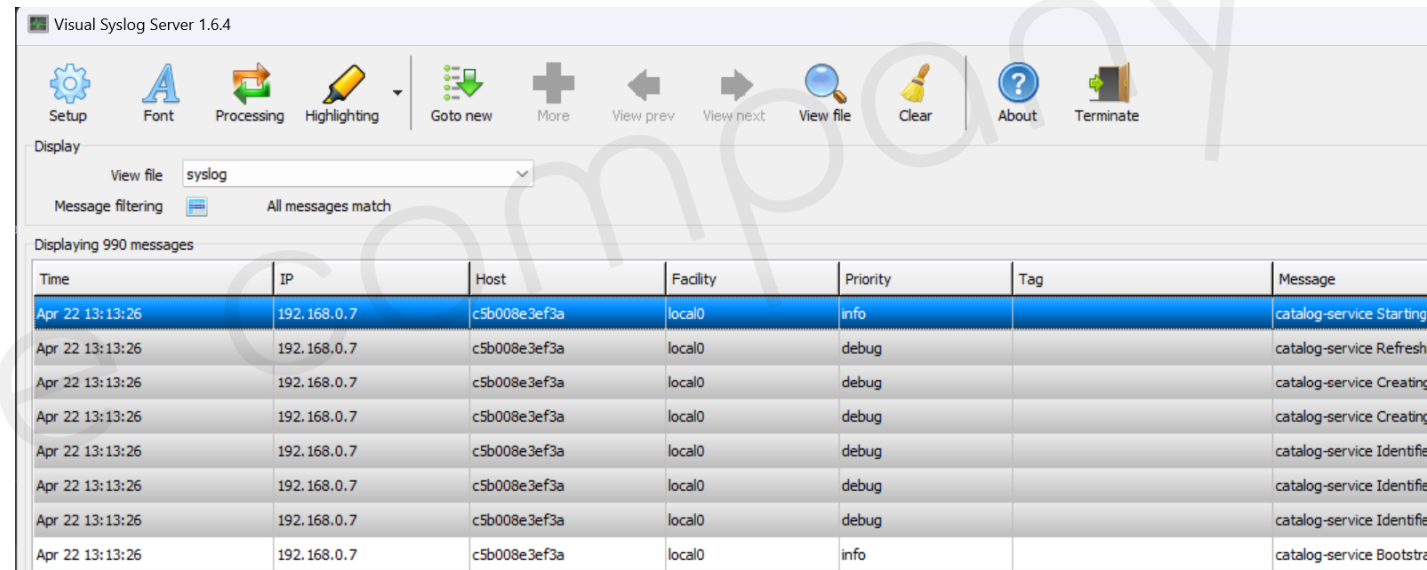
Visual Syslog Server 1.5.0

Display: View file: syslog, Message filtering: All messages match, the last 1.0 Mb of the 316.7 Mb

Displaying 7361 messages of 7369

| Time            | IP          | Host   | Facility | Priority        | Tag                                                                  | Message |
|-----------------|-------------|--------|----------|-----------------|----------------------------------------------------------------------|---------|
| Dec 18 11:22:32 | 192.168.1.2 | daemon | err      | tinyproxy[9409] | readbuff: recv() error "Connection reset by peer"                    |         |
| Dec 18 11:22:33 | 192.168.1.2 | daemon | err      | tinyproxy[9456] | read_request_line: Client (file descriptor: 5) closed                |         |
| Dec 18 11:22:33 | 192.168.1.2 | daemon | err      | tinyproxy[9456] | Error reading readable client_fd 5                                   |         |
| Dec 18 11:22:33 | 192.168.1.2 | host1  | daemon   | warning         | tinyproxy[9456] Could not retrieve request entity                    |         |
| Dec 18 11:22:35 | 192.168.1.1 | host1  | mail     | warning         | postfix/smtpd[5589] warning: hostname 93-179-64-10.jp-msk-ix-p3.ru c |         |
| Dec 18 11:22:35 | 192.168.1.1 | host1  | mail     | info            | postfix/smtpd[5589] connect from unknown[93.179.64.10]               |         |
| Dec 18 11:22:36 | 192.168.1.1 | host1  | mail     | info            | postfix/smtpd[5589] NOQUEUE: reject: RCPT from unknown[93.179.6      |         |
| Dec 18 11:22:36 | 192.168.1.1 | host1  | mail     | info            | postfix/smtpd[5589] disconnect from unknown[93.179.64.10]            |         |

UDP 0.0.0.0:514 TCP 0.0.0.0:514 [1]



The screenshot shows the Visual Syslog Server 1.6.4 application interface. The title bar is "Visual Syslog Server 1.6.4". The interface has a menu bar with "Setup", "Font", "Processing", "Highlighting", "Goto new", "More", "View prev", "View next", "View file", "Clear", "About", and "Terminate". Below the menu bar, there is a "Display" section with a "View file" dropdown set to "syslog" and a "Message filtering" section with "All messages match". Below this, it says "Displaying 990 messages".

| Time            | IP          | Host         | Facility | Priority | Tag | Message                  |
|-----------------|-------------|--------------|----------|----------|-----|--------------------------|
| Apr 22 13:13:26 | 192.168.0.7 | c5b008e3ef3a | local0   | info     |     | catalog-service Starting |
| Apr 22 13:13:26 | 192.168.0.7 | c5b008e3ef3a | local0   | debug    |     | catalog-service Refresh  |
| Apr 22 13:13:26 | 192.168.0.7 | c5b008e3ef3a | local0   | debug    |     | catalog-service Creating |
| Apr 22 13:13:26 | 192.168.0.7 | c5b008e3ef3a | local0   | debug    |     | catalog-service Creating |
| Apr 22 13:13:26 | 192.168.0.7 | c5b008e3ef3a | local0   | debug    |     | catalog-service Identify |
| Apr 22 13:13:26 | 192.168.0.7 | c5b008e3ef3a | local0   | debug    |     | catalog-service Identify |
| Apr 22 13:13:26 | 192.168.0.7 | c5b008e3ef3a | local0   | debug    |     | catalog-service Identify |
| Apr 22 13:13:26 | 192.168.0.7 | c5b008e3ef3a | local0   | info     |     | catalog-service Bootstr  |

# Docker Container Logging

## Logging (Windows)

### ■ Syslog 서버 설치 필요

- Windows에서는 Syslog를 사용할 수 없기 때문에, 별도의 Syslog Server 설치

```
$ docker run -d -p 13306:3306 --log-driver syslog \
    --log-opt syslog-address=udp://192.168.0.7:514 \
    -e MARIADB_ALLOW_EMPTY_ROOT_PASSWORD=true \
    --name mariadb1 \
    mariadb:latest
```

| syslog 파일 위치 → C:\W<USER\_HOME>\WAppData\WLocal\Wvisualsyslog\Wsyslog

```
syslog
C: > Users > zitan > AppData > Local > visualsyslog > syslog
1 192.168.0.7 Apr 22 11:54:13 daemon info c7507dee37c6[1298] 2024-04-22 11:54:13+00:00 [Note] [Entrypoint]: Entrypoint script for MariaDB Server 1:10.11.2+maria~ubu2204 started.
2 192.168.0.7 Apr 22 11:54:13 daemon info c7507dee37c6[1298] 2024-04-22 11:54:13+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
3 192.168.0.7 Apr 22 11:54:13 daemon info c7507dee37c6[1298] 2024-04-22 11:54:13+00:00 [Note] [Entrypoint]: Entrypoint script for MariaDB Server 1:10.11.2+maria~ubu2204 started.
4 192.168.0.7 Apr 22 11:54:13 daemon info c7507dee37c6[1298] 2024-04-22 11:54:13+00:00 [Note] [Entrypoint]: Initializing database files
5 192.168.0.7 Apr 22 11:54:13 daemon err c7507dee37c6[1298] 2024-04-22 11:54:13 0 [Warning] mariabdd: io_uring_queue_init() failed with ENOSYS: check seccomp filters, and the k
6 192.168.0.7 Apr 22 11:54:13 daemon err c7507dee37c6[1298] 2024-04-22 11:54:13 0 [Warning] InnoDB: liburing disabled: falling back to innodb_use_native_aio=OFF
7 192.168.0.7 Apr 22 11:54:16 daemon info c7507dee37c6[1298] PLEASE REMEMBER TO SET A PASSWORD FOR THE MariaDB root USER !
8 192.168.0.7 Apr 22 11:54:16 daemon info c7507dee37c6[1298] To do so, start the server, then issue the following command:
9 192.168.0.7 Apr 22 11:54:16 daemon info c7507dee37c6[1298] '/usr/bin/mariadb-secure-installation'
10 192.168.0.7 Apr 22 11:54:16 daemon info c7507dee37c6[1298] which will also give you the option of removing the test
11 192.168.0.7 Apr 22 11:54:16 daemon info c7507dee37c6[1298] databases and anonymous user created by default. This is
12 192.168.0.7 Apr 22 11:54:16 daemon info c7507dee37c6[1298] strongly recommended for production servers.
13 192.168.0.7 Apr 22 11:54:16 daemon info c7507dee37c6[1298] See the MariaDB Knowledgebase at https://mariadb.com/kb
```

# Docker Container Logging

## Logging (Windows)

### ■ Syslog 서버 설치 필요

- my-backend 예제 사용 → section8-win branch

■ <https://github.com/joneconsulting/devops-docker/tree/main/my-backend>

```
$ docker-compose --f docker-compose-win.yml up -d
```

```
version: '3.7'
services:
  my-backend:
    image: edowon0623/catalog-service:syslog-ng-v1.3
    environment:
      - spring.datasource.url=jdbc:mariadb://my-db:3306/mydb
      - spring.datasource.password=
      - DOCKER-COMPOSE-SYSLOG-ADDRESS=192.168.0.7
      - DOCKER-COMPOSE-SYSLOG-PORT=514
    ports:
      - 8088:8088
    depends_on:
      - my-db
    networks:
      - my-network
```

■ Windows PC ip-address

```
<configuration debug="true" scan="true" scanPeriod="30 seconds">
  <property name="SYSLOG-ADDRESS" value="${DOCKER-COMPOSE-SYSLOG-ADDRESS}"/>
  <property name="SYSLOG-PORT" value="${DOCKER-COMPOSE-SYSLOG-PORT}"/>

  <appender name="SYSLOG" class="ch.qos.logback.classic.net.SyslogAppender">
    <syslogHost>${SYSLOG-ADDRESS}</syslogHost>
    <facility>LOCAL0</facility>
    <port>${SYSLOG-PORT}</port>
    <throwableExcluded>true</throwableExcluded>
    <suffixPattern>catalog-service %m thread:%t priority:%p category:%c
    exception:%exception</suffixPattern>
  </appender>
```

# Docker Container Logging

## Logging (Windows)

- Spring Boot Application + MariaDB → Syslog

Visual Syslog Server 1.6.4

Setup Font Processing Highlighting Goto new More View prev View next View file Clear About Terminate

Display

View file: syslog [the last 1.0 Mb of the 1.0 Mb]

Message filtering: ☒ Text contains "called"

Displaying 3 messages of 3049

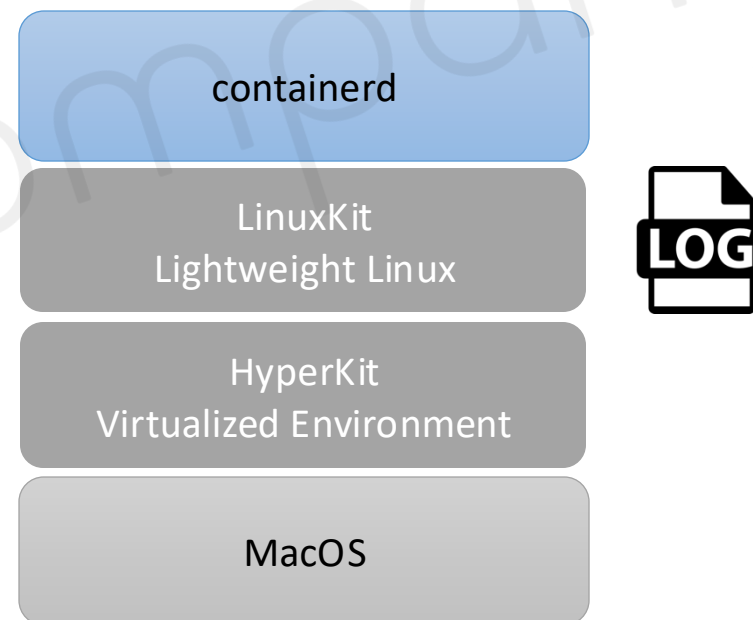
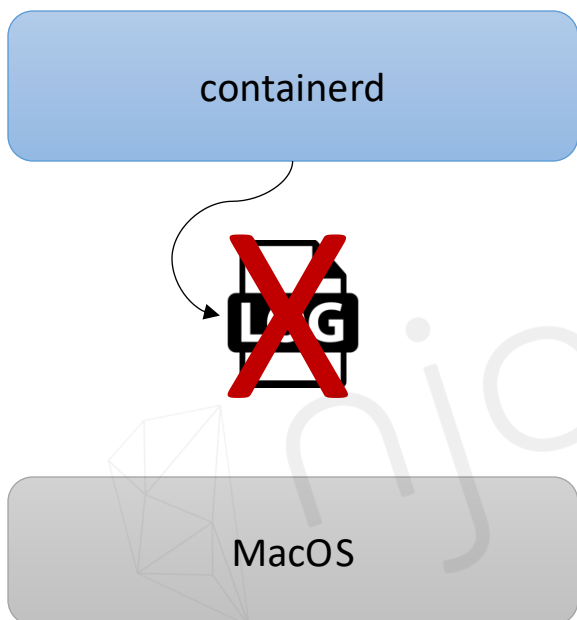
| Time            | IP          | Host         | Facility | Priority | Tag | Message                                                                                                                                                  |
|-----------------|-------------|--------------|----------|----------|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Apr 22 21:48:08 | 192.168.0.7 | rog-flow     | local0   | debug    |     | catalog-service called catalogs thread:http-nio-8088-exec-2 priority:DEBUG category:com.example.catalogservice.controller.CatalogController exception:   |
| Apr 22 21:52:59 | 192.168.0.7 | rog-flow     | local0   | debug    |     | catalog-service called catalogs thread:http-nio-8088-exec-5 priority:DEBUG category:com.example.catalogservice.controller.CatalogController exception:   |
| Apr 22 13:07:19 | 192.168.0.7 | ac387661012e | local0   | debug    |     | catalog-service [called catalogs] thread:http-nio-8088-exec-2 priority:DEBUG category:com.example.catalogservice.controller.CatalogController exception: |

# Docker Container Logging

## Logging (MacOS)

- 컨테이너 로그 파일 확인

- | `docker inspect` → "LogPath": `"/var/lib/docker/containers/~`
- | `/var/lib/docker/containers/${CONTAINER_ID}/${CONTAINER_ID}-json.log`



# Docker Container Logging

## Logging (MacOS)

- MacOS Apple chip 에서는 /var/lib/docker/containers/~ 의 내용 확인 어려움

<https://gist.github.com/BretFisher/5e1a0c7bcc4c735e716abf62afad389>

```
$ docker run -it --rm --privileged --pid=host justincormack/nsenter1  
  
# cd /var/lib/docker/containers  
  
# ls -al
```

```
▶ docker run -it --rm --privileged --pid=host justincormack/nsenter1  
~ # cd /var/lib/docker/containers  
/var/lib/docker/containers # ls -al  
total 16  
drwx--x---  4 root    root    4096 Apr 25 08:55 .  
drwx--x--- 12 root    root    4096 Apr 25 07:51 ..  
drwx--x---  4 root    root    4096 Apr 25 08:41 b8423bbc4892bcaa0dbce80562f8038f4ab0ca4f9f0d23206e875eb0e4368cdf  
drwx--x---  4 root    root    4096 Apr 25 08:55 c9fbbc4457fde5b6c3df3b71a13fdf33e7e599c32abaec519d34d503e89a304d
```

# Docker Container Logging

## Logging (MacOS)

- 컨테이너 로그 파일 → syslog

| syslog → 유닉스 계열 운영체제에서 로그 수집하는 도구 (커널, 보안, 애플리케이션 로그 등)

| <https://github.com/joneconsulting/devops-docker.git> → section8 branch

```
$ cd my-brackend
```

```
$ docker-compose -f docker-compose.yml up syslog-ng -d
```

```
$ docker-compose ps
```

```
services:
  syslog-ng:
    image: linuxserver/syslog-ng:latest
    container_name: syslog-ng
    environment:
      - PUID=1000
      - PGID=1000
      - TZ=Etc/UTC
    volumes:
      - ./syslog-ng/config:/config
      - ./syslog-ng/log:/var/log #optional
    ports:
      - 514:5514/udp
```

# Docker Container Logging

## Logging (MacOS)

- Mariadb → Syslog

| mariadb container에서 syslog-ng container로 로그 출력

```
$ docker run -d -p 13306:3306 --log-driver syslog \
--log-opt syslog-address=udp://172.17.0.1:514 \
-e MARIADB_ALLOW_EMPTY_ROOT_PASSWORD=true \
--name mariadb1 \
mariadb:latest
```

```
Apr 29 13:30:19 172.20.0.1 bd7663fe5b59[311]: 2024-04-29 13:30:19 0 [Note] InnoDB: Buffer pool(s) load completed at 240429 13:30:19
Apr 29 13:30:19 172.20.0.1 bd7663fe5b59[311]: 2024-04-29 13:30:19 0 [Note] Server socket created on IP: '0.0.0.0'.
Apr 29 13:30:19 172.20.0.1 bd7663fe5b59[311]: 2024-04-29 13:30:19 0 [Note] Server socket created on IP: '::'.
Apr 29 13:30:19 172.20.0.1 bd7663fe5b59[311]: 2024-04-29 13:30:19 0 [Note] mariadbd: Event Scheduler: Loaded 0 events
Apr 29 13:30:19 172.20.0.1 bd7663fe5b59[311]: 2024-04-29 13:30:19 0 [Note] mariadbd: ready for connections.
Apr 29 13:30:19 172.20.0.1 bd7663fe5b59[311]: Version: '11.3.2-MariaDB-1:11.3.2+maria~ubu2204' socket: '/run/mysqld/mysqld.sock' port: 3306 mariadb.org binary distribution
Apr 29 22:30:37 192.168.65.1 AddressBookSourceSync[82513]:
Apr 29 22:30:40 192.168.65.1 AddressBookManager[82546]:
Apr 29 22:30:51 192.168.65.1 AddressBookManager[82612]:
Apr 29 22:31:48 192.168.65.1 powerd[99]: [System: PrevIdle PrevDisp DeclUser kDisp]
```

# Docker Container Logging

## Logging Demo

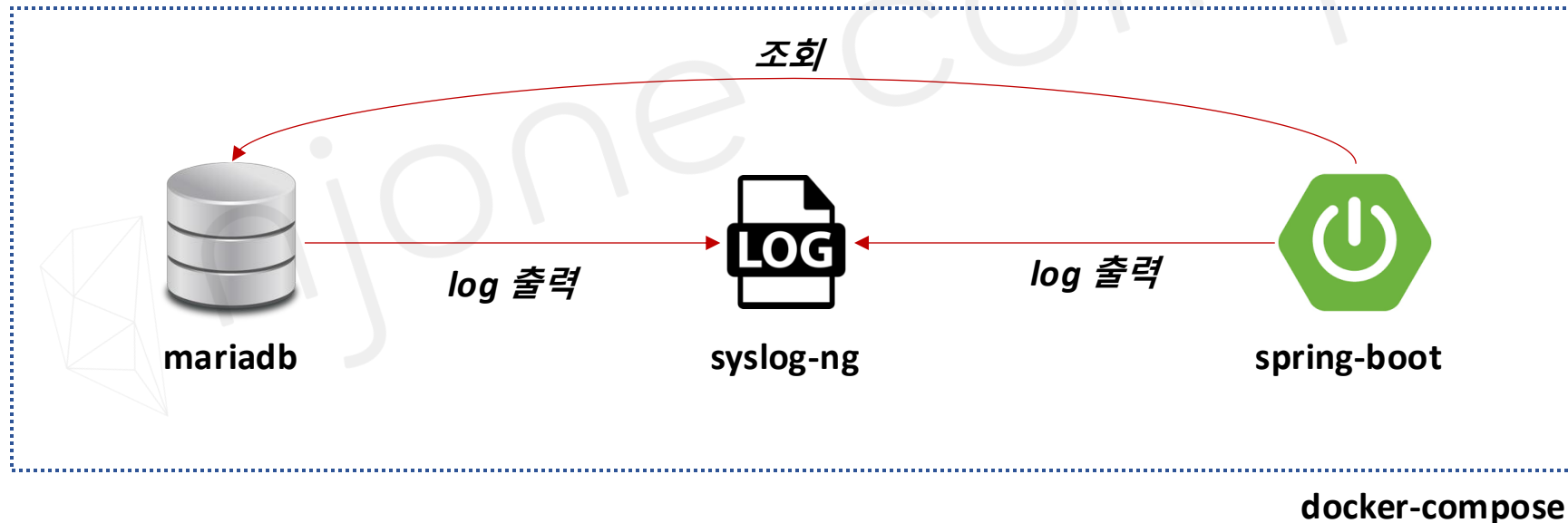
- syslog-ng + mariadb + my-backend

| docker-compose로 syslog-ng / mariadb / my-backend 서비스 실행

| <https://github.com/joneconsulting/devops-docker.git> → section8 branch

```
$ cd my-backend
```

```
$ docker-compose -f docker-compose-app.yml up -d
```



## Monitoring

- 일반 Monitoring → 서버의 CPU, 메모리 사용량, 디스크 공간 등의 기본 지표를 추적
- 컨테이너 Monitoring → 컨테이너의 생명 주기가 짧고, 여러 개의 컨테이너 관리 필요
- Prometheus
  - | 컨테이너 환경의 애플리케이션을 모니터링 하기 위한 오픈소스 시스템
  - | 여러 컨테이너 상태를 개별적으로 추적 (분산 애플리케이션 모니터링 가능)
  - | 컨테이너 상태에 대한 Metrics 수집 → 외부 서비스와 연계

# Docker Container Monitoring

## Prometheus (Linux)

- /etc/docker/daemon.json 파일 수정

```
{  
  "metrics-addr": "0.0.0.0:9323",  
  "experimental": true  
}
```

| Docker 재실행

| Docker Prometheus 실행

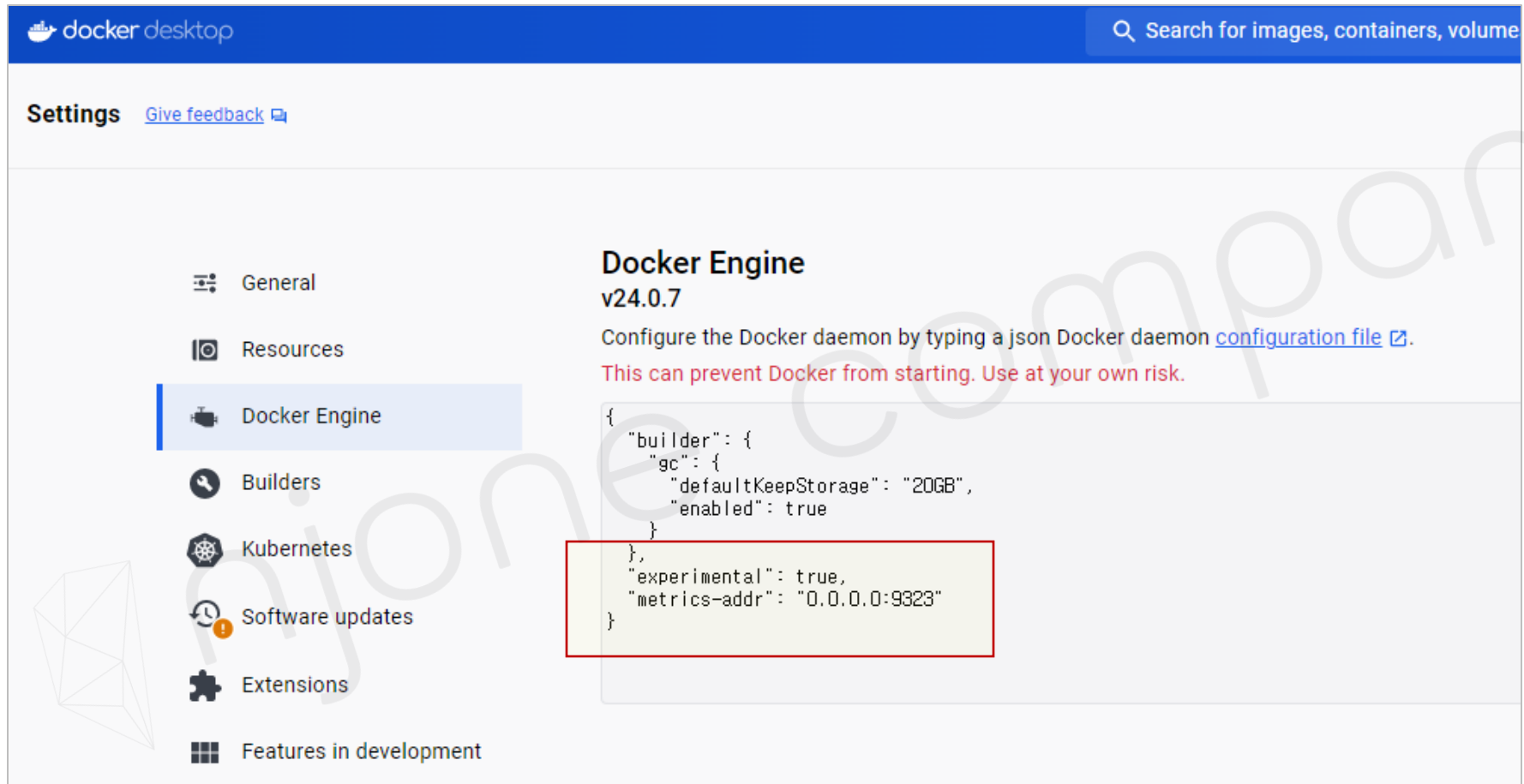
```
$ hostIP=172.31.5.111
```

```
$ docker container run -e DOCKER_HOST=$hostIP -d --rm -p 9090:9090 diamol/prometheus:2.13.1
```

# Docker Container Monitoring

## Prometheus (Windows & MacOS)

- Docker Desktop > Settings > Docker Engine → Docker Desktop 재실행



# Docker Container Monitoring

## Prometheus (Windows & MacOS)

- Docker Prometheus 실행

| Windows)

```
C:\> set hostIP=[IP Address]
```

```
C:\> docker container run -e DOCKER_HOST=%hostIP% -d --rm -p 9090:9090 diamol/prometheus:2.13.1
```

| MacOS)

```
$ hostIP=[IP Address]
```

```
$ docker container run -e DOCKER_HOST=$hostIP -d --rm -p 9090:9090 diamol/prometheus:2.13.1
```

# Docker Container Monitoring

## Prometheus

- Prometheus 실행

```
← → ↻ ⚠ 주의 요약 43.203.199.147:9090/targets
← → ↻ ⚠ 주의 요약 43.203.199.147:9323/metrics

# HELP builder_builds_failed_total Number of failed image builds
# TYPE builder_builds_failed_total counter
builder_builds_failed_total{reason="build_canceled"} 0
builder_builds_failed_total{reason="build_target_not_reachable_error"} 0
builder_builds_failed_total{reason="command_not_supported_error"} 0
builder_builds_failed_total{reason="dockerfile_empty_error"} 0
builder_builds_failed_total{reason="dockerfile_syntax_error"} 0
builder_builds_failed_total{reason="error_processing_commands_error"} 0
builder_builds_failed_total{reason="missing_onbuild_arguments_error"} 0
builder_builds_failed_total{reason="unknown_instruction_error"} 0
# HELP builder_builds_triggered_total Number of triggered image builds
# TYPE builder_builds_triggered_total counter
builder_builds_triggered_total 0
# HELP engine_daemon_container_actions_seconds The number of seconds it takes to process each container action
# TYPE engine_daemon_container_actions_seconds histogram
engine_daemon_container_actions_seconds_bucket{action="changes",le="0.005"} 1
engine_daemon_container_actions_seconds_bucket{action="changes",le="0.01"} 1
engine_daemon_container_actions_seconds_bucket{action="changes",le="0.025"} 1
engine_daemon_container_actions_seconds_bucket{action="changes",le="0.05"} 1
```

# Docker Container Monitoring

## Prometheus

- Mariadb 컨테이너 실행

```
← → ↺ ⚠ 주의 요함 43.203.199.147:9323/metrics
engine_daemon_container_actions_seconds_bucket{action="start",le="0.25"} 1
engine_daemon_container_actions_seconds_bucket{action="start",le="0.5"} 6
engine_daemon_container_actions_seconds_bucket{action="start",le="1"} 6
engine_daemon_container_actions_seconds_bucket{action="start",le="2.5"} 6
engine_daemon_container_actions_seconds_bucket{action="start",le="5"} 6
engine_daemon_container_actions_seconds_bucket{action="start",le="10"} 6
engine_daemon_container_actions_seconds_bucket{action="start",le="+Inf"} 6
engine_daemon_container_actions_seconds_sum{action="start"} 1.5138992470000001
engine_daemon_container_actions_seconds_count{action="start"} 6
# HELP engine_daemon_container_states_containers The count of containers in various states
# TYPE engine_daemon_container_states_containers gauge
engine_daemon_container_states_containers{state="paused"} 0
engine_daemon_container_states_containers{state="running"} 2
engine_daemon_container_states_containers{state="stopped"} 0
# HELP engine_daemon_engine_cpus_cpus The number of cpus that the host system of the engine has
# TYPE engine_daemon_engine_cpus_cpus gauge
engine_daemon_engine_cpus_cpus 1
```

# Docker Container Monitoring

## Prometheus

- Graph

Prometheus Alerts Graph Status ▾ Help

☐ Enable query history

engine\_daemon\_container\_states\_containers{state="running"}

Execute

- insert metric at cursor · ⬆ ⬇ ⬇ ⬆

Graph

Console



Moment



Element

engine\_daemon\_container\_states\_containers{instance="DOCKER\_HOST:9323",job="docker",state="running"}